

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

"Ingeniería de Software"

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA

Introducción a la Programación Proyecto Final:

**CHAGO ACCESS — Sistema Inteligente de
Control de Acceso y Conducta Universitaria**

**Nombres: Diego Zaid Baruch Baltazar, Gael Antonio Vega
Baez Sebastian Ugarte Aguilar**

Nombre Maestro: Santiago Esparza Guerrero Grado y grupo:

2do A Fecha de entrega: 19/05/2026



MATRÍCULAS: 42102356, 43420907, 4210247

ÍNDICE

1. Portada
2. Introducción
3. Descripción del programa
4. Requerimientos funcionales
5. Requerimientos no funcionales
6. Problema a resolver
7. Entradas / procesos / salidas
8. Herramientas utilizadas
9. Código versión estudiante (casero)
10. Código versión mejorada con IA
11. Pruebas del sistema
12. Evidencias visuales (espacios)
13. Conclusión

PROYECTO FINAL INTRODUCCIÓN A LA PROGRAMACIÓN

CHAGO ACCESS

Sistema Inteligente de Control de Acceso y Conducta Universitaria

1. INTRODUCCIÓN

Actualmente en muchos espacios universitarios el acceso a laboratorios, talleres y áreas académicas todavía se realiza de forma manual, generando problemas como pérdida de tiempo, registros incompletos y poca organización.

Pensando en una solución sencilla pero útil, se desarrolló **CHAGOACCESS**, un sistema elaborado en Python orientado al control de acceso universitario.

Este programa permite registrar estudiantes que ingresan al laboratorio mediante un menú interactivo donde se almacena información básica como nombre, matrícula, programa académico, hora de ingreso y validaciones relacionadas con reglamentos internos.

Además de controlar entradas, el sistema incorpora reglas de conducta y cumplimiento institucional para generar reportes automáticos y estadísticas básicas.

El desarrollo del proyecto permitió aplicar conocimientos vistos durante la materia de Introducción a la Programación como:

- Variables
- Condicionales
- Ciclos
- Funciones
- Listas
- Archivos
- Validaciones
- Diseño lógico
- Documentación de código

Este proyecto busca representar una solución sencilla pero aplicable a situaciones reales dentro del entorno universitario.

2. DESCRIPCIÓN DEL PROGRAMA

2.1 ¿Qué hace el programa?

CHAGOACCESS es un sistema que permite administrar el acceso de estudiantes a un laboratorio universitario mediante el registro y validación de ciertos requisitos.

El usuario encargado podrá:

- Registrar estudiantes
 - Ver lista de accesos
 - Consultar estadísticas
 - Generar reportes
 - Detectar incumplimientos
 - Guardar historial
-

2.2 Finalidad del programa

La finalidad del sistema es mejorar el control de acceso y organización dentro de espacios universitarios utilizando herramientas básicas de programación.

También busca reducir errores humanos y facilitar el seguimiento de registros.

2.3 ¿Qué problema busca resolver?

En muchos laboratorios el control de entrada se realiza anotando nombres manualmente.

Esto ocasiona:

- Información incompleta
- Pérdida de registros
- Falta de seguimiento
- Retrasos
- Dificultad para generar estadísticas

El sistema automatiza ese proceso.

2.4 Requerimientos Funcionales

El sistema deberá:

1. Registrar estudiante.
2. Registrar matrícula.
3. Registrar carrera.
4. Registrar hora de entrada.

5. Validar credencial.
 6. Validar cumplimiento de reglamento.
 7. Mostrar lista de ingresos.
 8. Mostrar estadísticas.
 9. Guardar historial.
 10. Permitir salir del sistema.
-

2.5 Requerimientos No Funcionales

1. Interfaz sencilla.
2. Fácil mantenimiento.
3. Código comentado.
4. Compatible con Python 3.
5. Respuesta rápida.
6. Fondo claro para evidencias.
7. Diseño entendible para estudiantes.

3. PROBLEMA A RESOLVER

3.1 Descripción del problema

Dentro de la universidad existen laboratorios donde el ingreso debe mantenerse organizado.

Actualmente el registro manual provoca errores y poca eficiencia.

Se propone desarrollar un sistema que permita automatizar el proceso de control de acceso mediante programación.

3.2 Entradas, Procesos y Salidas

Entradas

- Nombre
- Matrícula
- Carrera
- Credencial (Sí/No)
- Bata (Sí/No)

Procesos

- Captura de datos
- Validación
- Almacenamiento
- Conteo
- Generación de estadísticas

Salidas

- Registro exitoso
- Reportes
- Estadísticas
- Historial

4. HERRAMIENTAS UTILIZADAS

4.1 Herramientas usadas

Lenguaje

Python

Entorno

Visual Studio Code

Herramientas de apoyo

- Flowgorithm
 - PSeInt
-

Inteligencia Artificial (apoyo)

Se utilizó inteligencia artificial únicamente como apoyo para:

- Organización del documento
- Optimización del código
- Generación de ideas

La lógica fue comprendida y adaptada por el estudiante.

5. PROGRAMA

5.1 Introducción al desarrollo

Para demostrar el proceso de aprendizaje se presentan dos versiones:

Versión 1

```
registros = []

while True:

    print("\n===== CHAGOACCESS =====")

    print("1. Registrar acceso")

    print("2. Ver registros")

    print("3. Estadísticas")

    print("4. Salir")

    op = input("Selecciona: ")

    if op == "1":

        nombre = input("Nombre: ")

        matricula = input("Matricula: ")

        carrera = input("Carrera: ")

        credencial = input("Trae credencial (si/no): ")

        bata = input("Trae bata (si/no): ")

        estado = "ACEPTADO"

        if credencial.lower() == "no":

            estado = "DENEGADO"
```

```
if bata.lower() == "no":
```

```
    estado = "DENEGADO"
```

```
datos = [
```

```
    nombre,
```

```
    matricula,
```

```
    carrera,
```

```
    credencial,
```

```
    bata,
```

```
    estado
```

```
]
```

```
registros.append(datos)
```

```
archivo = open("historial.txt","a")
```

```
archivo.write(
```

```
    nombre+
```

```
    " | "+
```

```
    matricula+
```

```
    " | "+
```

```
    estado+
```

```
    "\n"
```

```
)
```

```
archivo.close()
```

```
print("\nRegistro guardado")
```

```
elif op=="2":
```

```
if len(registros)==0:
```

```
    print("Sin registros")
```

```
else:
```

```
    for x in registros:
```

```
        print("-----")
```

```
        print("Nombre:",x[0])
```

```
        print("Matricula:",x[1])
```

```
        print("Carrera:",x[2])
```

```
        print("Credencial:",x[3])
```

```
        print("Bata:",x[4])
```

```
        print("Estado:",x[5])
```

```
elif op=="3":
```

```
    total=len(registros)
```

```
    aceptados=0
```

```
    for x in registros:
```

```
        if x[5]=="ACEPTADO":
```

```
            aceptados+=1
```

```
    print("Total:",total)
```

```
    print("Aceptados:",aceptados)
```

```
    print("Denegados:",total-aceptados)
```

```
elif op=="4":
```

```
    print("Programa terminado")
```

```
    break
```

```
else:
```

```
    print("Opcion incorrecta")
```

Versión 2

Código mejorado utilizando buenas prácticas y apoyo de IA.

```
from tkinter import *
from tkinter import messagebox
from datetime import datetime

registros=[]

def guardar(reg):

    with open(

        "historial_chagoaccess.txt",

        "a",

        encoding="utf8"

    ) as f:
```

```
f.write(

f"""

Nombre: {reg["nombre"]}

Matricula: {reg["matricula"]}

Carrera: {reg["carrera"]}

Hora: {reg["hora"]}

Estado: {reg["estado"]}

"""

)

def registrar():

    nombre=e_nombre.get().strip()

    matricula=e_matricula.get().strip()

    carrera=e_carrera.get().strip()

    if not nombre or not matricula or not carrera:

        messagebox.showwarning(

            "Error",
```

```
        "Completa todos los campos"

    )

    return

hora=datetime.now().strftime("%H:%M")

estado="AUTORIZADO"

if (

    not credencial_var.get()

    or not bata_var.get()

):

    estado="DENEGADO"

dato={

    "nombre":nombre,

    "matricula":matricula,

    "carrera":carrera,
```

```
        "hora":hora,

        "estado":estado

    }

registros.append(dato)

guardar(dato)

lista.insert(

    END,

    f"{nombre} → {estado}"

)

actualizar()

limpiar()

messagebox.showinfo(

    "CHAGOACCESS",

    f"Acceso {estado}"

)
```

```
def actualizar():

    total=len(registros)

    buenos=0

    for r in registros:

        if r["estado"]=="AUTORIZADO":

            buenos+=1

    stats.config(

text=f"""
Total: {total}

Autorizados: {buenos}

Denegados: {total-buenos}
"""

    )
```



```
def limpiar():

    e_nombre.delete(0,END)

    e_matricula.delete(0,END)

    e_carrera.delete(0,END)

    credencial_var.set(True)

    bata_var.set(True)


root=Tk()

root.title(

    "🕷️ CHAGOACCESS - SPIDER EDITION"

)

root.geometry(

    "1000x650"
```

```
)

root.config(

bg="#0b132b"

)

titulo=Label(

root,

text="🕷 CHAGOACCESS",

font=(

"Impact",

32

),

bg="#0b132b",

fg="#ff3333"

)
```

```
titulo.pack(
```

```
    pady=15
```

```
)
```

```
panel=Frame(
```

```
    root,
```

```
    bg="#16324f",
```

```
    padx=20,
```

```
    pady=20
```

```
)
```

```
panel.pack()
```

```
Label(
```

```
    panel,
```

```
text="Nombre",
```

```
bg="#16324f",
```

```
fg="white"
```

```
) .grid(
```

```
row=0,
```

```
column=0
```

```
)
```

```
e_nombre=Entry(
```

```
panel,
```

```
width=30
```

```
)
```

```
e_nombre.grid(
```

```
row=0,
```

```
column=1
```

```
)
```

```
Label(
```

```
panel,  
  
text="Matricula",  
  
bg="#16324f",  
  
fg="white"  
  
) .grid(  
row=1,  
column=0  
)  
  
e_matricula=Entry(  
panel,  
width=30  
)  
  
e_matricula.grid(  
row=1,  
column=1  
)
```

```
Label(  
  
panel,  
  
text="Carrera",  
  
bg="#16324f",  
  
fg="white"  
  
) .grid(  
row=2,  
column=0  
)  
  
e_carrera=Entry(  
panel,  
width=30  
)  
  
e_carrera.grid(  
row=2,  
column=1  
)
```

```
credencial_var=BooleanVar()
```

```
credencial_var.set(True)
```

```
bata_var=BooleanVar()
```

```
bata_var.set(True)
```

```
Checkbutton(
```

```
panel,
```

```
text="Trae Credencial",
```

```
variable=credencial_var,
```

```
bg="#16324f",
```

```
fg="white",
```

```
selectcolor="#16324f"
```

```
) .grid(  
row=3,  
column=1,  
sticky="w"  
)
```

```
Checkbutton(  

```

```
panel,
```

```
text="Trae Bata",
```

```
variable=bata_var,
```

```
bg="#16324f",
```

```
fg="white",
```

```
selectcolor="#16324f"
```

```
) .grid(  

```



```
row=4,

column=1,

sticky="w"

)


def ver_historial():

    try:

        with open(

            "historial_chagoaccess.txt",

            "r",

            encoding="utf8"

        ) as f:

            contenido=f.read()

            ventana=Toplevel()

            ventana.title(

                "Historial"

            )
```

```
ventana.geometry(  
    "700x500"  
)  
  
texto=Text(  
    ventana  
)  
  
texto.pack(  
    fill="both",  
    expand=True  
)  
  
texto.insert(  
    END,  
    contenido  
)  
  
except:  
  
    messagebox.showinfo(  
        "Historial",  
        "No hay registros"  
    )
```

```
def reiniciar_historial():

    confirmar=messagebox.askyesno(

        "Reiniciar",

        "¿Seguro que quieres borrar TODO el historial?"

    )

    if confirmar:

        global registros

        registros=[]

        lista.delete(

            0,

            END

        )

        stats.config(

            text="Total: 0"

        )
```

```
with open(  
    "historial_chagoaccess.txt",  
    "w",  
    encoding="utf8"  
    ) as f:
```

```
    f.write("")
```

```
messagebox.showinfo(  
    "Listo",  
    "Historial reiniciado"
```

```
)
```

```
Button(  
    root,
```

```
    text="🎲 REGISTRAR",  
    font=(  
        "Arial",  
        14  
    ),
```

```
width=22,

bg="#d62828",

fg="white",

command=registrar

).pack(

    pady=15

)

Button(

root,

text="📋 VER HISTORIAL",

font=(

"Arial",

12

),
```

```
bg="#00509d",

fg="white",

width=22,

command=ver_historial

).pack(

    pady=5

)

Button(

    root,

    text="🗑 REINICIAR HISTORIAL",

    font=("Arial",12),

    bg="#8b0000",

    fg="white",

    width=22,
```

```
command=reiniciar_historial
```

```
) .pack(
```

```
pady=5
```

```
)
```

```
lista=Listbox(
```

```
root,
```

```
width=80,
```

```
height=12,
```

```
bg="#1b263b",
```

```
fg="white"
```

```
)
```

```
lista.pack()
```

```
stats=Label(  
  
root,  
  
text="Total: 0",  
  
bg="#0b132b",  
  
fg="#58a6ff",  
  
font=(  
    "Arial",  
    14  
)  
)  
  
stats.pack(  
    pady=20  
)  
  
Label(  

```



```
root,  
  
text="Friendly Neighborhood Access System",  
  
bg="#0b132b",  
  
fg="gray"  
  
) .pack()  
  
root.mainloop()
```

6. PRUEBAS DEL SISTEMA

Caso	Entrada	Resultado esperado	Resultado obtenido
1	Credencial=S Bata=S	Acceso autorizado	Correcto

2	Credencial=N	Acceso denegado	Correcto
3	Bata=N	Acceso denegado	Correcto
4	Ver registros	Mostrar historial	Correcto
5	Estadísticas	Mostrar conteos	Correcto

7. EVIDENCIA VISUAL

Pantalla 1 — Menú principal

The screenshot shows the main menu of the CHAGOACCESS SPIDER EDITION system. The interface is dark-themed with a central form for user registration and login. The form includes fields for 'Nombre', 'Matricula', and 'Carrera', followed by checkboxes for 'Trae Credencial' and 'Trae Bata'. Below the form are three buttons: 'REGISTRAR' (red), 'VER HISTORIAL' (blue), and 'REINICIAR HISTORIAL' (red). A large empty box is present below the buttons, and at the bottom, it shows 'Total: 0' and the text 'Friendly Neighborhood Access System'.

CHAGOACCESS — SPIDER EDITION

CHAGOACCESS

Nombre
Matricula
Carrera
☒ Trae Credencial
☒ Trae Bata

REGISTRAR

VER HISTORIAL

REINICIAR HISTORIAL

Total: 0

Friendly Neighborhood Access System

Se muestran las opciones principales del sistema.

Pantalla 2 — Registro



CHAGOACCESS

Nombre
Matricula
Carrera

☒ Trae Credencial
☒ Trae Bata

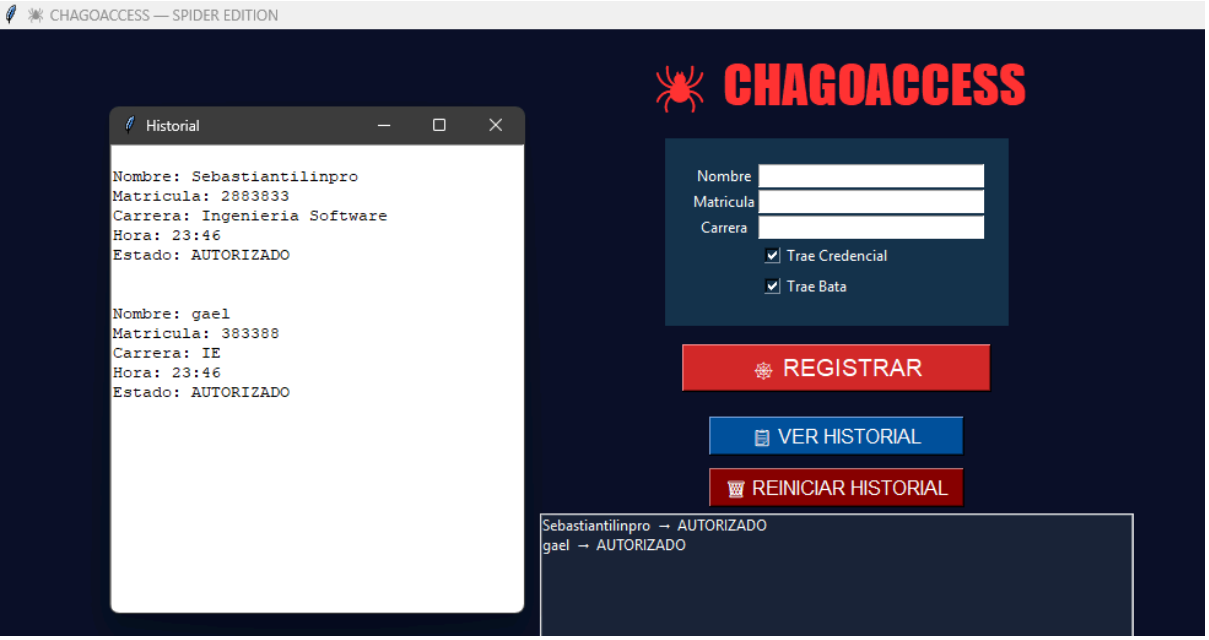
REGISTRAR

VER HISTORIAL

REINICIAR HISTORIAL

Captura de información del usuario.

Pantalla 3 — Historial



CHAGOACCESS — SPIDER EDITION

CHAGOACCESS

Nombre
Matricula
Carrera

☒ Trae Credencial
☒ Trae Bata

REGISTRAR

VER HISTORIAL

REINICIAR HISTORIAL

Historial

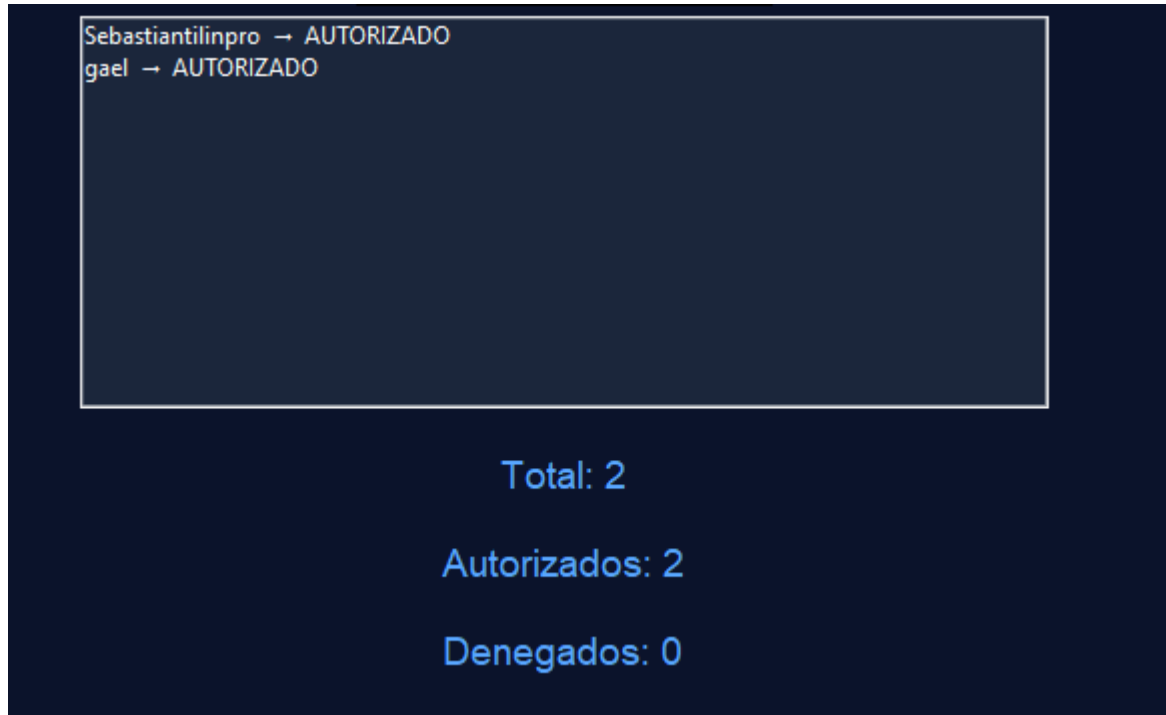
Nombre: Sebastiantilinpro
Matricula: 2883833
Carrera: Ingenieria Software
Hora: 23:46
Estado: AUTORIZADO

Nombre: gael
Matricula: 383388
Carrera: IE
Hora: 23:46
Estado: AUTORIZADO

Sebastiantilinpro → AUTORIZADO
gael → AUTORIZADO

Visualización de datos almacenados.

Pantalla 4 — Estadísticas



Conteo de accesos autorizados y rechazados.

8. CONCLUSIONES

Durante el desarrollo de este proyecto se logró aplicar conocimientos adquiridos durante la materia de Introducción a la Programación.

Se comprendió cómo resolver un problema cotidiano mediante el uso de estructuras de control, listas, funciones y almacenamiento de información.

Una de las dificultades encontradas fue organizar correctamente el flujo del programa y validar datos para evitar errores durante la ejecución.

Estas dificultades fueron resueltas realizando pruebas constantes y mejorando la lógica del código.

También se observó que herramientas modernas como la inteligencia artificial pueden apoyar el desarrollo, siempre y cuando exista comprensión del funcionamiento del código.

Este proyecto permitió entender que la programación no consiste únicamente en escribir instrucciones, sino en construir soluciones útiles para problemas reales.